tf.compat.v1.distribute.StrategyExtended Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/distribute/StrategyExtended

> *Additional APIs for algorithms that need to be distribution-aware.*

Function Signatures

```
tf.compat.v1.distribute.StrategyExtended( container_strategy
)
```

Code Examples

```
tf.compat.v1.distribute.StrategyExtended(
container_strategy
)
```

```
tf.compat.v1.distribute.StrategyExtended(
container_strategy
)
```

```
batch_reduce_to(
reduce_op, value_destination_pairs, options=None
)
```


```
batch_reduce_to(
reduce_op, value_destination_pairs, options=None
)
```


```
broadcast_to(
tensor, destinations
)
```


```
broadcast_to(
tensor, destinations
)
```


```
call_for_each_replica(
fn, args=(), kwargs=None
)
```


```
call_for_each_replica(
fn, args=(), kwargs=None
)
```


Documentation

Additional APIs for algorithms that need to be distribution-aware.

Inherits From: StrategyExtended

Some common use cases of functions on this page:

- Locality

tf.distribute.DistributedValues can have the same locality as a distributed variable, which leads to a mirrored value residing on the same devices as the variable (as opposed to the compute devices). Such values may be passed to a call to tf.distribute.StrategyExtended.update to update the value of a variable. You may use tf.distribute.StrategyExtended.colocate_vars_with to give a variable the same locality as another variable. You may convert a "PerReplica" value to a variable's locality by using tf.distribute.StrategyExtended.reduce_to or tf.distribute.StrategyExtended.batch_reduce_to.

- How to update a distributed variable

A distributed variable is variables created on multiple devices. As discussed in the glossary,
mirrored variable and SyncOnRead variable are two examples. The standard pattern for updating distributed variables is to:

- In your function passed to tf.distribute.Strategy.run,

compute a list of (update, variable) pairs. For example, the update might be a gradient of the loss with respect to the variable.

- Switch to cross-replica mode by calling

tf.distribute.get_replica_context().merge_call() with the updates and variables as arguments.

- Call

tf.distribute.StrategyExtended.reduce_to(VariableAggregation.SUM, t, v)
(for one variable) or tf.distribute.StrategyExtended.batch_reduce_to
(for a list of variables) to sum the updates.

- Call tf.distribute.StrategyExtended.update(v) for each variable to update

its value.

Steps 2 through 4 are done automatically by class
tf.keras.optimizers.Optimizer if you call its
tf.keras.optimizers.Optimizer.apply_gradients method in a replica context.

In fact, a higher-level solution to update a distributed variable is by
calling assign on the variable as you would do to a regular tf.Variable.
You can call the method in both replica context and cross-replica context.
For a mirrored variable, calling assign in replica context requires you
to specify the aggregation type in the variable constructor. In that case,
the context switching and sync described in steps 2 through 4 are handled for
you. If you call assign on mirrored variable in cross-replica context,
you can only assign a single value or assign values from another mirrored
variable or a mirrored tf.distribute.DistributedValues. For a SyncOnRead
variable, in replica context, you can simply call assign on it and no
aggregation happens under the hood. In cross-replica context, you can only
assign a single value to a SyncOnRead variable. One example case is restoring
from a checkpoint: if the aggregation type of the variable is
tf.VariableAggregation.SUM, it is assumed that replica values were added
before checkpointing, so at the time of restoring, the value is divided by
the number of replicas and then assigned to each replica; if the aggregation
type is tf.VariableAggregation.MEAN, the value is assigned to each replica

directly.

Attributes

This is expected to return a constant value that will not be changed throughout its life cycle.

Methods

batch_reduce_to

View source

Combine multiple reduce_to calls into one for faster execution.

Similar to reduce_to, but accepts a list of (value, destinations) pairs. It's more efficient than reduce each value separately.

This API currently can only be called in cross-replica context. Other variants to reduce values across replicas are:

- tf.distribute.StrategyExtended.reduce_to: the non-batch version of

this API.

- tf.distribute.ReplicaContext.all_reduce: the counterpart of this API

in replica context. It supports both batched and non-batched all-reduce.

- tf.distribute.Strategy.reduce: a more convenient method to reduce

to the host in cross-replica context.

See reduce_to for more information.

@tf.function
def step_fn(var):

def merge_fn(strategy, value, var):
# All-reduce the value. Note that value here is a
# tf.distribute.DistributedValues.
reduced = strategy.extended.batch_reduce_to(
tf.distribute.ReduceOp.SUM, [(value, var)])[0]
strategy.extended.update(var, lambda var, value: var.assign(value),
args=(reduced,))

value = tf.identity(1.)
tf.distribute.get_replica_context().merge_call(merge_fn,
args=(value, var))

def run(strategy):
with strategy.scope():
v = tf.Variable(0.)
strategy.run(step_fn, args=(v,))
return v

run(tf.distribute.MirroredStrategy(["GPU:0", "GPU:1"]))
MirroredVariable:{
0: ,
1:
}
run(tf.distribute.experimental.CentralStorageStrategy(
compute_devices=["GPU:0", "GPU:1"], parameter_device="CPU:0"))

run(tf.distribute.OneDeviceStrategy("GPU:0"))

broadcast_to

View source

Mirror a tensor on one device to all worker devices.

call_for_each_replica

View source

Run fn once per replica.

fn may call tf.get_replica_context() to access methods such as replica_id_in_sync_group and merge_call().

merge_call() is used to communicate between the replicas and re-enter the cross-replica context. All replicas pause their execution having encountered a merge_call() call. After that the merge_fn-function is executed. Its results are then unwrapped and given back to each replica call. After that execution resumes until fn is complete or encounters another merge_call(). Example:

colocate_vars_with

View source

Scope that controls which devices variables will be created on.

No operations should be added to the graph inside this scope, it

should only be used when creating variables (some implementations work by changing variable creation, others work by using a tf.compat.v1.colocate_with() scope).

This may only be used inside self.scope().

Example usage:

experimental_make_numpy_dataset

View source

Makes a dataset for input provided via a numpy array.

This avoids adding numpy_input as a large constant in the graph, and copies the data to the machine or machines that will be p